B.Tech CSE - Third Year | Lab 15

Assignment 15: Subsets and Arrangements • *Aim: Generate subsets and arrangements using recursion/backtracking.*

Experiment No. 15.1: Print all subsets of {1, 2, 3}

1. **Aim:** Implement the task specified in the BCS2024545 lab assignment.
2. **Program:**

```
public class Task_01_subset_of_array {
    static int[] a = {1, 2, 3};
    static boolean[] chosen = new boolean[a.length];
    static void subsets(int i) {
        if (i == a.length) {
            System.out.print("{");
            boolean first = true;
            for (int j = 0; j < a.length; j++) {
                if (chosen[j]) {
                    if (!first) System.out.print(", ");
                    System.out.print(a[j]);
                    first = false;
                }
            }
            System.out.println("}");
            return;
        }
        chosen[i] = false;
        subsets(i + 1);
        chosen[i] = true;
        subsets(i + 1);
    }
    public static void main(String[] args) {
        subsets(0);
    }
}
```

3. **Output:** `{} {3} {2} {2, 3} {1} {1, 3} {1, 2} {1, 2, 3}`
4. **Viva-Voce:** What is a subset? How many subsets does a set of n elements have? Why are include and exclude choices both needed?

Experiment No. 15.2: Print arrangements of a, b and c

1. **Aim:** Implement the task specified in the BCS2024545 lab assignment.
2. **Program:**

```
public class Task_02_Print_arrangements_of_a_b_and_c {
    static char[] letters = {'a', 'b', 'c'};
    static boolean[] used = new boolean[letters.length];
    static char[] result = new char[letters.length];
    static void arrange(int pos) {
        if (pos == letters.length) {
            System.out.println(new String(result));
            return;
        }
        for (int i = 0; i < letters.length; i++) {
            if (!used[i]) {
                used[i] = true;
                result[pos] = letters[i];
                arrange(pos + 1);
                used[i] = false;
            }
        }
    }
    public static void main(String[] args) {
        arrange(0);
    }
}
```

3. **Output:** `abc acb bac bca cab cba`
4. **Viva-Voce:** What is a permutation? Why is used required? How many arrangements exist for three distinct letters?

Experiment No. 15.3: Print arrangements of x and y

1. **Aim:** Implement the task specified in the BCS2024545 lab assignment.
2. **Program:**

```
public class Task_03_Print_arrangements_of_x_and_y {
    static char[] letters = {'x', 'y'};
    static boolean[] used = new boolean[letters.length];
    static char[] result = new char[letters.length];
    static void arrange(int pos) {
        if (pos == letters.length) {
            System.out.println(new String(result));
            return;
        }
        for (int i = 0; i < letters.length; i++) {
            if (!used[i]) {
                used[i] = true;
                result[pos] = letters[i];
                arrange(pos + 1);
                used[i] = false;
            }
        }
    }
    public static void main(String[] args) {
        arrange(0);
    }
}
```

3. **Output:** `xy yx`
4. **Viva-Voce:** How many permutations of two distinct letters exist? What is backtracking? Why unmark a letter after recursion?